

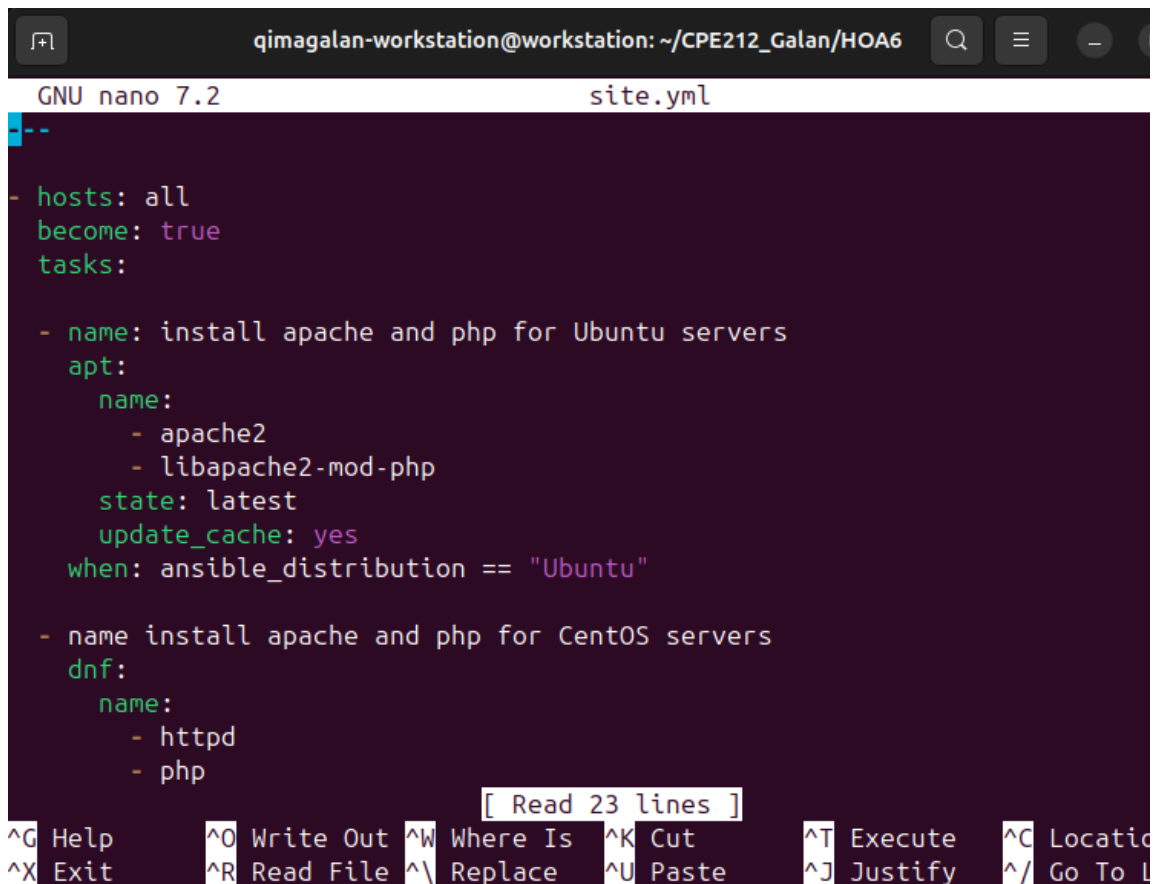
COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	09/1/2026	DATE SUBMITTED:	09/3/2026
NAME:	Galan, Irick Marvin	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 6 Targeting Specific Nodes

1. PROCEDURE OUTPUT

Task 1: Targeting Specific Nodes

1. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.



```

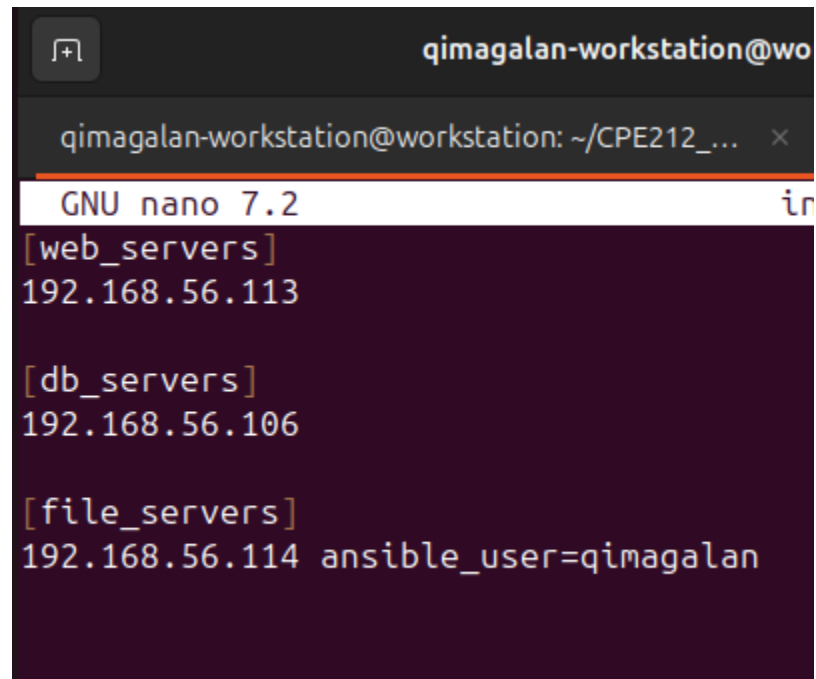
qimagalan-workstation@workstation: ~/CPE212_Galan/HOA6
GNU nano 7.2 site.yml
--
- hosts: all
  become: true
  tasks:
    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"
    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php

```

[Read 23 lines]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To L

2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:



```
qimagalan-workstation@workstation: ~/CPE212_... x
GNU nano 7.2
[web_servers]
192.168.56.113

[db_servers]
192.168.56.106

[file_servers]
192.168.56.114 ansible_user=qimagalan
```

Make sure to save the file and exit.

Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.

3. Edit the *site.yml* by following the image below:

qimagalan-workstation@workstation: ~/CPE212_... × qimagalan-wo

GNU nano 7.2

site.yml

```
---
- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"
```

```
- hosts: web_servers
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      tags: apache, centos, httpd
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

The *pre-tasks* command tells the ansible to run it before any other thing. In the *pre-tasks*, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at *web_servers*. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.

Run the *site.yml* file and describe the result.

So far, we defined one main task that only executes for the webservers group and that is the installation of apache and php. It is also important that we take note that each task block has a separate '- hosts' variable, because the dash signifies the start of an independent task with separate variables, allowing us to reuse the key variable names such as hosts and become without any conflict. We will notice that the task counterpart for the CentOS was skipped, it is because I assigned the webservers group to contain only one Ubuntu. I only made use of three remote servers, one per each group, and only the file_servers group has the CentOS machine.

```
qimagalan-workstation@workstation: ~/CPE212_Galan/HOA6
qimagalan-workstation@workstation: ~/CPE212_... x qimagalan-workstation@workstation: ~/CPE212_... x v
qimagalan-workstation@workstation: ~/CPE212_Galan/HOA6$ ansible-playbook site.yml -i
inventory.yml -K
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]
ok: [192.168.56.113]
ok: [192.168.56.114]

TASK [install updates (CentOS)] *****
skipping: [192.168.56.113]
skipping: [192.168.56.106]
ok: [192.168.56.114]

TASK [install updates (Ubuntu)] *****
skipping: [192.168.56.114]
ok: [192.168.56.106]
ok: [192.168.56.113]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.113]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.113]

PLAY RECAP *****
192.168.56.106      : ok=2    changed=0    unreachable=0    failed=0    skippe
ed=1    rescued=0    ignored=0
192.168.56.113    : ok=4    changed=0    unreachable=0    failed=0    skippe
```

- Let's try to edit again the *site.yml* file. This time, we are going to add plays targeting the other servers. This time we target the *db_servers* by adding it on the current *site.yml*. Below is an example: (Note add this at the end of the playbooks from task 1.3.

```
- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      yum:
        name: mariadb-server
        state: latest
        when: ansible_distribution == "CentOS"

    - name: "Mariadb- Restarting/Enabling"
      service:
        name: mysql
        state: restarted
        enabled: true

    - name: install mariadb package (Ubuntu)
      apt:
        name: mariadb-server
        state: latest
        when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.

Run the *site.yml* file and describe the result.

The db_servers also contain another Ubuntu so the CentOS counterpart of the task was skipped, however the third play crashed. It will not work because if we look at the sequence of the ansible plays executed, we are restarting mariadb before it is even installed.

```
qimagalan-workstation@workstation: ~/CPE212_... x qimagalan-workstation@workstation: ~/CPE212_... x v
qimagalan-workstation@workstation:~/CPE212_Galan/H0A6$ ansible-playbook site.yml -i
inventory.yml -K
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]
ok: [192.168.56.106]
ok: [192.168.56.114]

TASK [install updates (CentOS)] *****
skipping: [192.168.56.113]
skipping: [192.168.56.106]
ok: [192.168.56.114]

TASK [install updates (Ubuntu)] *****
skipping: [192.168.56.114]
ok: [192.168.56.113]
ok: [192.168.56.106]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.113]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.113]
```

```
PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [Mariadb- Restarting/Enabling] *****
fatal: [192.168.56.106]: FAILED! => {"changed": false, "msg": "Could not find the re
quested service mysql: host"}

PLAY RECAP *****
192.168.56.106      : ok=3    changed=0    unreachable=0    failed=1    skipp
ed=2    rescued=0    ignored=0
192.168.56.113     : ok=4    changed=0    unreachable=0    failed=0    skipp
ed=2    rescued=0    ignored=0
192.168.56.114     : ok=2    changed=0    unreachable=0    failed=0    skipp
ed=1    rescued=0    ignored=0

qimagalan-workstation@workstation:~/CPE212_Galan/H0A6$
```

- Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: `systemctl status mariadb`. Do this on the CentOS server also.

The mariadb was not installed despite running the playbook since it crashed while trying to restart the mariadb service which did not exist yet at that point in time.

```
qimagalan-workstation@server2: ~
qimagalan-workstation@server2:~$ ip address | grep enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    inet 192.168.56.106/24 brd 192.168.56.255 scope global dynamic noprefixroute enp
0s8
qimagalan-workstation@server2:~$ sudo systemctl status mariadb
[sudo] password for qimagalan-workstation:
Unit mariadb.service could not be found.
qimagalan-workstation@server2:~$
```

Applying the Fix:

```
- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      dnf:
        use_backend: dnf5
        name: mariadb-server
        state: latest
        when: ansible_distribution == "CentOS"

    - name: install mariadb package (Ubuntu)
      apt:
        name: mariadb-server
        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: "Mariadb- Restarting/Enabling"
      service:
        name: mysql
        state: restarted
        enabled: true
```

```
qimagalan-workstation@workstation: ~/CPE212_... x qimagalan-workstation@workstation: ~/CPE212_... x
qimagalan-workstation@workstation:~/CPE212_Galan/H0A6$ ansible-playbook site.yml -i
inventory.yml -K
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]
ok: [192.168.56.106]
ok: [192.168.56.114]

TASK [install updates (CentOS)] *****
skipping: [192.168.56.113]
skipping: [192.168.56.106]
ok: [192.168.56.114]

TASK [install updates (Ubuntu)] *****
skipping: [192.168.56.114]
ok: [192.168.56.106]
ok: [192.168.56.113]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.113]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.113]
```

```
PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [install mariadb package (Ubuntu)] *****
changed: [192.168.56.106]

TASK [Mariadb- Restarting/Enabling] *****
changed: [192.168.56.106]

PLAY RECAP *****
192.168.56.106      : ok=5    changed=2    unreachable=0    failed=0    skippe
ed=2    rescued=0    ignored=0
192.168.56.113     : ok=4    changed=0    unreachable=0    failed=0    skippe
ed=2    rescued=0    ignored=0
192.168.56.114     : ok=2    changed=0    unreachable=0    failed=0    skippe
ed=1    rescued=0    ignored=0

qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$
```

Status of mariadb after running the fixed playbook:

```
qimagalan-workstation@server2:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.11.14 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enab
   Active: active (running) since Tue 2026-09-01 21:55:23 PST; 47s ago
     Docs: man:mariabdd(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 77298 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/ru
   Process: 77300 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_START>
   Process: 77302 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR>
   Process: 77375 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_STAR>
   Process: 77377 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUC>
   Main PID: 77362 (mariabdd)
    Status: "Taking your SQL requests now..."
       Tasks: 13 (limit: 14914)
      Memory: 78.7M (peak: 81.8M)
         CPU: 1.042s
        CGroup: /system.slice/mariadb.service
                └─77362 /usr/sbin/mariabdd

Sep 01 21:55:23 server2 mariabdd[77362]: 2026-09-01 21:55:23 0 [Note] InnoDB: log s>
Sep 01 21:55:23 server2 mariabdd[77362]: 2026-09-01 21:55:23 0 [Note] Plugin 'FEEDB>
Sep 01 21:55:23 server2 mariabdd[77362]: 2026-09-01 21:55:23 0 [Note] InnoDB: Loadi>
Sep 01 21:55:23 server2 mariabdd[77362]: 2026-09-01 21:55:23 0 [Warning] You need t>
Sep 01 21:55:23 server2 mariabdd[77362]: 2026-09-01 21:55:23 0 [Note] Server socket>
Sep 01 21:55:23 server2 mariabdd[77362]: 2026-09-01 21:55:23 0 [Note] InnoDB: Buffe>
Sep 01 21:55:23 server2 mariabdd[77362]: 2026-09-01 21:55:23 0 [Note] /usr/sbin/mar>
Sep 01 21:55:23 server2 mariabdd[77362]: Version: '10.11.14-MariaDB-0ubuntu0.24.04.>
Sep 01 21:55:23 server2 systemd[1]: Started mariadb.service - MariaDB 10.11.14 data>
Sep 01 21:55:23 server2 /etc/mysql/debian-start[77380]: Upgrading MariaDB tables if>
lines 1-28/28 (END)
```

```
qimagalan@localhost:~
[qimagalan@localhost ~]$ sudo systemctl status mariadb
[sudo] password for qimagalan:
Unit mariadb.service could not be found.
[qimagalan@localhost ~]$
```

The CentOS was skipped because it is not part of the `db_servers` group and therefore as expected, there should be no `mariadb` service that exists.

6. Edit the `site.yml` again. This time we will append the code to configure installation on the `file_servers` group. We can add the following on our file.

```
hosts: file_servers
become: true
tasks:

- name: install samba package
  package:
    name: samba
    state: latest
```

Make sure to save the file and exit.

Run the `site.yml` file and describe the result.

In the screenshot, I simply focused on the play number that installs samba we can see that it successfully ran, however, since I have run the play before the screenshot was made it shows that 192.168.56.114 simply shows an 'ok' status instead of changed.

```
PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.114]

TASK [install samba package] *****
ok: [192.168.56.114]

PLAY RECAP *****
192.168.56.106      : ok=5    changed=1    unreachable=0    failed=0    skippe
ed=2    rescued=0    ignored=0
192.168.56.113      : ok=4    changed=0    unreachable=0    failed=0    skippe
ed=2    rescued=0    ignored=0
192.168.56.114      : ok=4    changed=0    unreachable=0    failed=0    skippe
ed=1    rescued=0    ignored=0

qinagalan-workstation@workstation:~/CPE212_Galan/H0A6$
```

The testing of the *file_servers* is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the *site.yml* file. Add tags to the playbook. After the name, we can place the tags: *name_of_tag*. This is an arbitrary command, which means you can use any name for a tag.

```
qinagalan-workstation@workstation:~/CPE212_Galan/H0A6$ cat site.yml
---
- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    tags: always
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    tags: always
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

hosts: web_servers
become: true
tasks:

- name: install apache and php for Ubuntu servers
  tags: apache, apache2, ubuntu
  apt:
    name:
      - apache2
      - libapache2-mod-php
    state: latest
  when: ansible_distribution == "Ubuntu"
```

```
- name: install apache and php for CentOS servers
tags: apache, centos, httpd
dnf:
  name:
    - httpd
    - php
  state: latest
when: ansible_distribution == "CentOS"

hosts: db_servers
become: true
tasks:

- name: install mariadb package (CentOS)
tags: centos, db, mariadb
yum:
  name: mariadb-server
  state: latest
when: ansible_distribution == "CentOS"

- name: install mariadb package (Ubuntu)
tags: db, mariadb, ubuntu
apt:
  name: mariadb-server
  state: latest
when: ansible_distribution == "Ubuntu"

- name: "Mariadb- Restarting/Enabling"
service:
  name: mysql
  state: restarted
  enabled: true
```

```
hosts: file_servers
become: true
tasks:

- name: install samba package
  tags: samba
  package:
    name: samba
    state: latest
```

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```
hosts: file_servers
become: true
tasks:

- name: start httpd
  tags: apache, centos, httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"
```

Figure 3.1.1

Make sure to save the file and exit.

You would also notice from our previous activity that we already created a module that runs a service.

```

TASK [start httpd] *****
ok: [192.168.56.114]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.106]

TASK [Mariadb- Restarting/Enabling] *****
changed: [192.168.56.106]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.114]

TASK [install samba package] *****
ok: [192.168.56.114]

PLAY RECAP *****
192.168.56.106      : ok=5    changed=1    unreachable=0    failed=0    skippe
ed=2    rescued=0    ignored=0
192.168.56.113      : ok=4    changed=0    unreachable=0    failed=0    skippe
ed=2    rescued=0    ignored=0
192.168.56.114      : ok=6    changed=0    unreachable=0    failed=0    skippe
ed=1    rescued=0    ignored=0

qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$

```

Figure 3.1.2

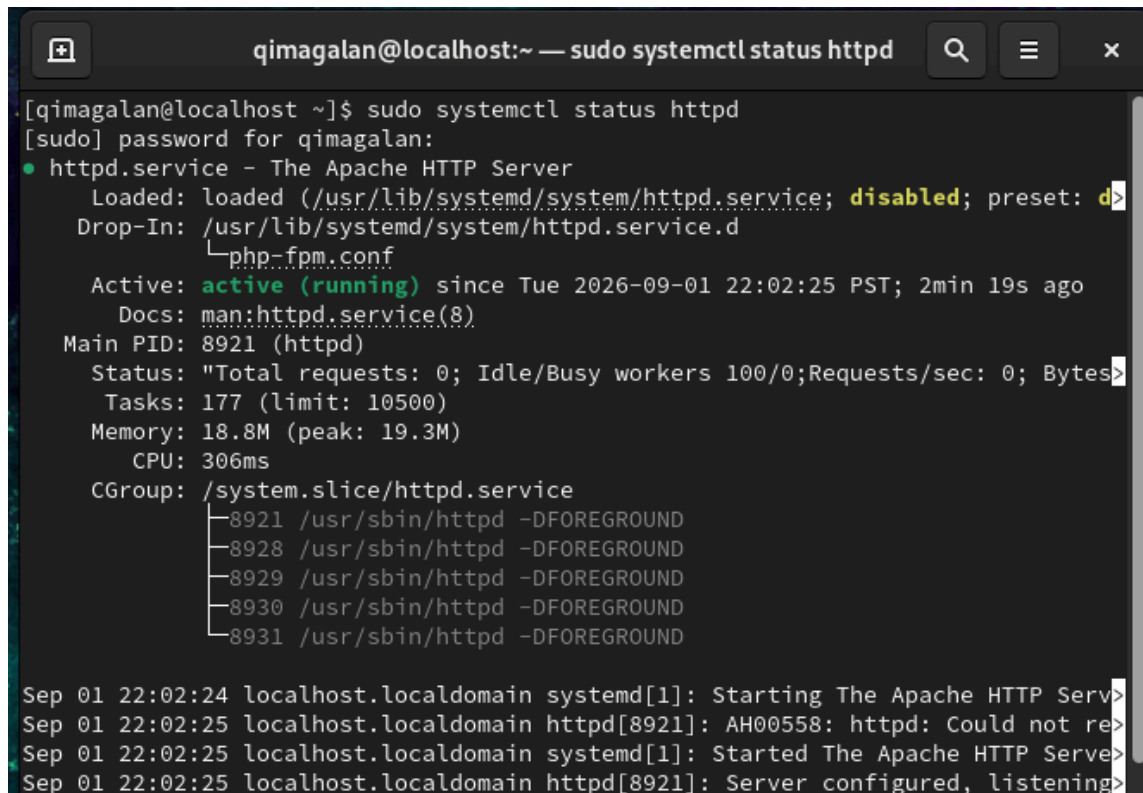
This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command `sudo systemctl stop httpd`. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.

After Running the `sudo systemctl stop httpd`

```
[qimagalan@localhost ~]$ sudo systemctl status httpd
[sudo] password for qimagalan:
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: c
   Drop-In: /usr/lib/systemd/system/httpd.service.d
           └─php-fpm.conf
   Active: inactive (dead)
     Docs: man:httpd.service(8)
lines 1-6/6 (END)
```

- Go to the local machine and this time, run the *site.yml* file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.
To automatically enable the service every time we run the playbook, use the command *enabled: true* similar to Figure 7.1.2 and save the playbook.



```
qimagalan@localhost:~ — sudo systemctl status httpd
[qimagalan@localhost ~]$ sudo systemctl status httpd
[sudo] password for qimagalan:
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: d>
   Drop-In: /usr/lib/systemd/system/httpd.service.d
           └─php-fpm.conf
   Active: active (running) since Tue 2026-09-01 22:02:25 PST; 2min 19s ago
     Docs: man:httpd.service(8)
  Main PID: 8921 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes>
      Tasks: 177 (limit: 10500)
     Memory: 18.8M (peak: 19.3M)
        CPU: 306ms
    CGroup: /system.slice/httpd.service
            └─8921 /usr/sbin/httpd -DFOREGROUND
              8928 /usr/sbin/httpd -DFOREGROUND
              8929 /usr/sbin/httpd -DFOREGROUND
              8930 /usr/sbin/httpd -DFOREGROUND
              8931 /usr/sbin/httpd -DFOREGROUND

Sep 01 22:02:24 localhost.localdomain systemd[1]: Starting The Apache HTTP Serv>
Sep 01 22:02:25 localhost.localdomain httpd[8921]: AH00558: httpd: Could not re>
Sep 01 22:02:25 localhost.localdomain systemd[1]: Started The Apache HTTP Serve>
Sep 01 22:02:25 localhost.localdomain httpd[8921]: Server configured, listening>
```

We could see that the CentOS server's *httpd* was definitely started, but it is disabled, and therefore every system boot the service requires to be started manually.

qimagalan-workstation@workstation: ~/CPE212_... x d

GNU nano 7.2

sit

```
- name: start httpd
tags: apache, centos, httpd
service:
  name: httpd
  state: started
  enabled: true
when: ansible_distribution == "CentOS"
```

Here we already added the enabled: true which allows the system to automatically start the httpd service upon system boot.

```

TASK [start httpd] *****
changed: [192.168.56.114]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.106]

TASK [Mariadb- Restarting/Enabling] *****
changed: [192.168.56.106]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.114]

TASK [install samba package] *****
ok: [192.168.56.114]

PLAY RECAP *****
192.168.56.106      : ok=5    changed=1    unreachable=0    failed=0
ed=2    rescued=0    ignored=0
192.168.56.113      : ok=4    changed=0    unreachable=0    failed=0
ed=2    rescued=0    ignored=0
192.168.56.114      : ok=6    changed=1    unreachable=0    failed=0
ed=1    rescued=0    ignored=0

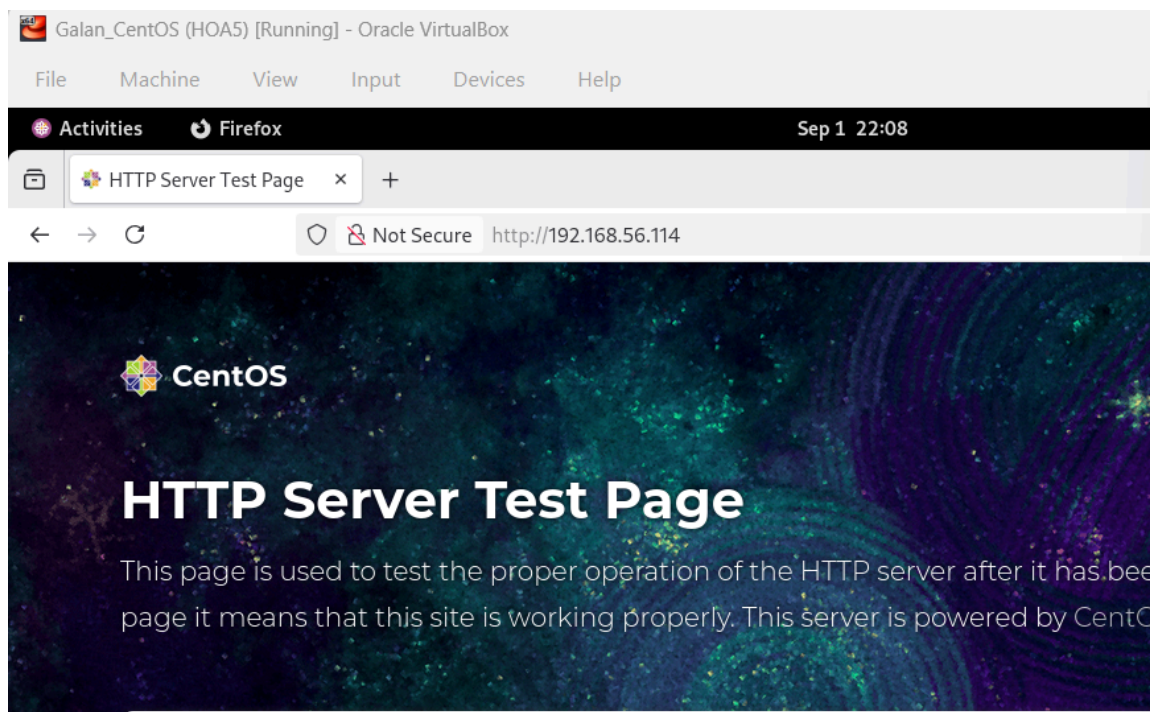
qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$

```

We can observe at the topmost part that the start httpd task shows a result of changed because it was moved from disabled state to enabled state.

```
qimagalan@localhost:~ — sudo systemctl status httpd
[qimagalan@localhost ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: active (running) since Tue 2026-09-01 22:02:25 PST; 4min 32s ago
     Docs: man:httpd.service(8)
   Main PID: 8921 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0"
   Tasks: 177 (limit: 10500)
  Memory: 18.8M (peak: 19.3M)
    CPU: 464ms
   CGroup: /system.slice/httpd.service
           └─8921 /usr/sbin/httpd -DFOREGROUND
             └─8928 /usr/sbin/httpd -DFOREGROUND
               └─8929 /usr/sbin/httpd -DFOREGROUND
                 └─8930 /usr/sbin/httpd -DFOREGROUND
                   └─8931 /usr/sbin/httpd -DFOREGROUND
```

Here in the systemctl status we can already see that the first 'disabled' keyword from earlier disappeared and was replaced with enabled.



The httpd was successfully started as it already popped up in the firefox upon entering the machine's own ip address.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:

1. Creating a Role for Web Servers

```
qimagalan-workstation@workstation: ~  
qimagalan-workstation@workstation:~$ ansible-galaxy init webservers_role --offline  
- Role webservers_role was created successfully  
qimagalan-workstation@workstation:~$
```

2. Creating a Role for Database Servers

```
qimagalan-workstation@workstation: ~  
qimagalan-workstation@workstation:~$ ansible-galaxy init dbservers_role --offline  
- Role dbservers_role was created successfully  
qimagalan-workstation@workstation:~$
```

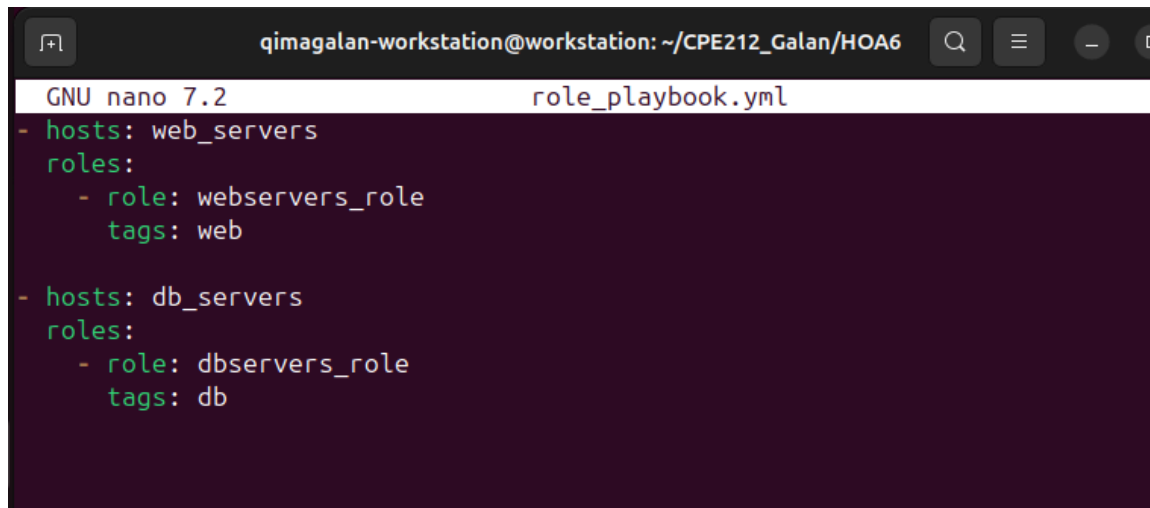
Although not shown here, we could also verify whether a role has been successfully created by using the `tree` command. The roles will be created in the current working directory, and since I mistakenly configured it there, I had to move it inside the CPE212 directory.

3. Combining Multiple Roles in a Playbook

```
qimagalan-workstation@workstation:~/CPE212_Galan/H0A6$ ansible-playbook role_playbook.yml -i inventory.yml --tags web  
ERROR! the role 'webservers_role' was not found in /home/qimagalan-workstation/CPE212_Galan/H0A6/roles:/home/qimagalan-workstation/.ansible/roles:/usr/share/ansible/roles:/etc/ansible/roles:/home/qimagalan-workstation/CPE212_Galan/H0A6  
  
The error appears to be in '/home/qimagalan-workstation/CPE212_Galan/H0A6/role_playbook.yml': line 3, column 7, but may  
be elsewhere in the file depending on the exact syntax problem.  
  
The offending line appears to be:  
  
    roles:  
    - role: webservers_role  
      ^ here
```

The first screenshot here shows an error, and I learned that the reason for it, is that ansible roles indicated in playbooks should be inside a 'roles' directory. As shown below, I created a new directory for roles in my H0A6 directory.

```
qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$ sudo nano role_playbook.
ml
[sudo] password for qimagalan-workstation:
qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$ ls
inventory.yml  role_playbook.yml  site.yml
qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$ cd ..
qimagalan-workstation@workstation:~/CPE212_Galan$ ls
ansible_roles  HOA4  HOA5  HOA6  inventory.yml  PrelimExam  README.md
qimagalan-workstation@workstation:~/CPE212_Galan$ mv ansible_roles roles
qimagalan-workstation@workstation:~/CPE212_Galan$ ls
HOA4  HOA5  HOA6  inventory.yml  PrelimExam  README.md  roles
qimagalan-workstation@workstation:~/CPE212_Galan$ mv roles ./HOA6
qimagalan-workstation@workstation:~/CPE212_Galan$ ls
HOA4  HOA5  HOA6  inventory.yml  PrelimExam  README.md
qimagalan-workstation@workstation:~/CPE212_Galan$ cd HOA6
qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$ ls
inventory.yml  role_playbook.yml  roles  site.yml
qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$
```



```
qimagalan-workstation@workstation: ~/CPE212_Galan/HOA6
GNU nano 7.2                                role_playbook.yml
- hosts: web_servers
  roles:
    - role: webservers_role
      tags: web
- hosts: db_servers
  roles:
    - role: dbservers_role
      tags: db
```

The screenshot above shows the simple playbook I made which simply prints a message, but the task for each machine is broken apart and stored in tasks/main.yml in their respective roles which is shown on the 2 screenshots below created under the webservers_role and dbservers_role respectively. The tasks directory under each role should directly contain the tasks name and only what to do. The consolidation of

plays is still done and maintained under a separate playbook.

```
qimagalan-workstation@workstation: ~/CPE212_Galan/HOA6/roles/webs...  
GNU nano 7.2 main.yml  
--  
- name: Simple messaging task in webserver Ubuntu  
  debug:  
    msg: "This is a task from the webserver_role at tasks/main.yml"
```

```
qimagalan-workstation@workstation: ~/CPE212_Galan/HOA6/roles/dbser...  
GNU nano 7.2 main.yml  
--  
- name: Simple Message Task in Ubuntu dbserver  
  debug:  
    msg: "This task is from dbserver_role under tasks/main.yml"
```

```
qimagalan-workstation@workstation: ~/CPE212_Galan/H0A6
qimagalan-workstation@workstation:~/CPE212_Galan/H0A6$ ansible-playbook role_playbook.yml -i inventory.yml --tags web

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]

TASK [webservers_role : Simple messaging task in webservers Ubuntu] *****
ok: [192.168.56.113] => {
  "msg": "This is a task from the webservers_role at tasks/main.yml"
}

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

PLAY RECAP *****
192.168.56.106      : ok=1    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.56.113     : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0

qimagalan-workstation@workstation:~/CPE212_Galan/H0A6$
```

As we have already defined the roles in the role_playbook.yml we only have to run the tagged roles by simply using `-tags <tags>` option with `ansible-playbook` in order to only run a specific role and their corresponding tasks.

```
qimagalan-workstation@workstation: ~/CPE212_Galan/HOA6
kipped=0    rescued=0    ignored=0

qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$ ansible-playbook roles/db_servers.yml -i inventory.yml --tags db

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [dbservers_role : Simple Message Task in Ubuntu dbservers] *****
ok: [192.168.56.106] => {
  "msg": "This task is from dbservers_role under tasks/main.yml"
}

PLAY RECAP *****
192.168.56.106      : ok=2    changed=0    unreachable=0    failed=0
kipped=0    rescued=0    ignored=0
192.168.56.113     : ok=1    changed=0    unreachable=0    failed=0
kipped=0    rescued=0    ignored=0

qimagalan-workstation@workstation:~/CPE212_Galan/HOA6$
```

To orchestrate multiple roles in a single playbook for different server groups.

Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

Reflections:

1. What is the importance of putting our remote servers into groups?
By placing remote servers into groups it permits a group targeted execution of a playbook. Ultimately this places a safeguard on our playbooks on which remote machine the effect and changes should be limited to.
2. What is the importance of tags in playbooks?
If through metagroups we specify the machines that should be altered and maintained by playbooks, the tags mimic that same effect but inside a playbook only. Instead of executing every single play in the playbook, we could simply execute specific blocks that are classified by the particular tag.

3. Why do you think some services need to be managed automatically in playbooks?

Playbook automation is not an option, it is a necessity. As we have seen from example metagroups that playbooks coordinate the different aspects of a deployed software such as web servers, database servers, and file servers. If a system administrator has to manually watch and monitor each and every one of the machines to look for potential security loopholes and failures, it would become unscalable as a system grows.

Conclusions

In this activity I have learned a lot of things which include, declaring task blocks with separate target hosts for each. I learned that the topmost dash symbol we typically see in '- hosts' declares a separate task block and sharing variable names between two independent task blocks is allowed, if we fail to properly format it, it will only pick the most recent value of the variable which became my first problem. Next, I also learned how to label each play with a tag that would allow us to run only the selected tasks that contain the specific tag, and this is applicable to the roles as well. Finally, the ansible roles are simply a template for creating cleaner plays. Since we separate the tasks and break them apart for each role. This increases maintainability since modifying one role will not affect another role and it also modularizes the code, and we simply have to use and consolidate the roles in a main playbook.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Claude

Purpose of Use:

As a guide for the supplementary number 3 'Combining multiple roles in a playbook'

Extent of Use:

AI served to provide some examples on how to use an ansible-role such as familiarizing with the structure of a role. The simple script is still my original work and the AI was only used to guide how to properly declare the roles syntax in the playbook.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.